

Security Assessment & Failure Handling Report

1. Security Assessment & API Review

API Endpoint Security

I secured protected API endpoints using token-based authentication. Public endpoints are limited to account registration and login, while wallet and transaction operations require a valid authentication token. Authorization checks ensure that an authenticated user can only perform operations associated with their own account.

Authentication and Authorization

Authentication is intentionally lightweight, as permitted by the assessment. A token is provided after successful login and must be supplied when accessing protected endpoints. Authentication middleware validates the token and identifies the requesting user. Authorization is then enforced within the application to prevent users from accessing or modifying another user's wallet or transactions.

Vulnerabilities and Mitigations

I considered common API and financial-application risks, including unauthorized access, SQL injection, invalid transactions, duplicate requests, race conditions, and exposure of sensitive information. Database operations are performed through Knex's query builder to reduce SQL injection risk. Financial operations are performed within database transactions so that related balance and transaction updates either succeed together or are rolled back. Requests that violate business rules, such as insufficient funds, invalid amounts, self-transfers, or invalid accounts, are rejected.

The onboarding flow also checks the Lendsqr Adjutor Karma blacklist before creating an account. A user who is identified as blacklisted is rejected and is not onboarded.

Input Validation and Backend Protection

I validate request data before executing business operations, including required fields, account identifiers, and transaction amounts. Amounts must be valid and greater than zero. Validation is reinforced by service-layer business rules so that critical checks cannot be bypassed by manipulating client requests.

Database constraints and transactional processing provide additional protection for wallet balances and transaction records. Errors are handled centrally to provide controlled responses without unnecessarily exposing internal application details.

Production Security Improvements

For a production deployment, I would introduce HTTPS/TLS, rate limiting, stronger request-schema validation, secure HTTP headers, stronger password policies, secure secret management, dependency vulnerability scanning, comprehensive audit logging, and more extensive security and integration testing. A production financial system would also require

appropriate KYC/AML controls, fraud detection, transaction monitoring, and continuous security monitoring.

2. Failure Handling & Debugging Assessment

Error Handling

Expected application failures are handled using controlled errors and centralized error-handling middleware. This provides consistent responses for authentication failures, invalid requests, insufficient funds, unavailable accounts, and other business-rule violations. Unexpected errors are handled without exposing sensitive implementation or database details.

Detection, Debugging and Tracing

When an issue occurs, I first reproduce the failure and trace the request through the application layers:

Route → Middleware → Controller → Service → Repository → Database

I use API testing and unit tests to verify both successful and failure scenarios. Transaction records and references can also be used to trace financial operations and investigate inconsistencies.

Logging, Monitoring and Reliability

The application should use structured logs containing useful diagnostic information such as request identifiers, endpoints, response status, execution time, and error details, while excluding passwords, tokens, credentials, and other sensitive data. In production, monitoring and alerting would be used to identify increased error rates, database failures, performance issues, and failed financial operations.

Database transactions and automated tests improve reliability by preventing partially completed financial operations and detecting regressions.

Example Failure Scenario

If a transfer fails after the sender's balance has been debited, I would trace the transaction, inspect the application logs and database records, and reproduce the request to determine where the failure occurred. I would specifically inspect the service logic, repository operations, and database transaction scope.

Because the transfer is executed within a database transaction, the expected behaviour is for the entire operation to roll back when a critical step fails. After identifying and fixing the cause, I would add a regression test for the failure scenario to ensure that the issue does not recur.